

Name : P. Samanvith

Batch No : 40

HT No : 2303A52090

Task 1: Weather API Integration

Task Description:

Connect to a Weather API to fetch current weather details for a given city. Implement proper error handling for invalid cities, unauthorized API keys, and network connection issues. Display the temperature, humidity, and weather description.

Prompt:

"Write a Python script using the requests library to fetch the current weather data for a user-provided city from the OpenWeatherMap API. Include robust error handling for invalid city names (HTTP 404), unauthorized/invalid API keys (HTTP 401), and general network connection errors. Extract and display the temperature in Celsius, humidity percentage, and a brief weather description."

Code:

```
import requests
import sys

def get_weather(city_name):
    # Replace with a valid OpenWeatherMap API key
    api_key = "YOUR_API_KEY_HERE"
    base_url =
    "[https://api.openweathermap.org/data/2.5/weather](https://api.openweathermap.org/data/2.5
    /weather)"

    params = {
        'q': city_name,
        'appid': api_key,
        'units': 'metric' # Fetch temperature in Celsius
    }

    try:
        response = requests.get(base_url, params=params, timeout=10)

        # Handle specific HTTP error codes
```

```

if response.status_code == 401:
    print("Error: Invalid API Key. Please check your credentials.")
    return
elif response.status_code == 404:
    print(f"Error: City '{city_name}' not found. Please check the spelling.")
    return

# Raise an exception for other bad status codes
response.raise_for_status()

data = response.json()

# Extract required information
temp = data['main']['temp']
humidity = data['main']['humidity']
description = data['weather'][0]['description'].capitalize()

print(f"--- Weather in {city_name.title()} ---")
print(f"Temperature : {temp}°C")
print(f"Humidity   : {humidity}%")
print(f"Conditions  : {description}")

except requests.exceptions.ConnectionError:
    print("Error: Failed to connect to the weather service. Check your internet connection.")
except requests.exceptions.Timeout:
    print("Error: The request timed out. Please try again later.")
except requests.exceptions.RequestException as e:
    print(f"An unexpected error occurred: {e}")

if __name__ == "__main__":
    city = input("Enter the city name: ")
    get_weather(city)

```

Output:

```

Enter the city name: Hyderabad
--- Weather in Hyderabad ---
Temperature : 32.5°C
Humidity   : 45%
Conditions  : Scattered clouds

```

Task 2: Currency Exchange Rates

Task Description:

Integrate a Currency Exchange API to convert INR to USD, EUR, and GBP. Validate user input for the amount and currency codes. Implement error handling for invalid responses and API downtime. Show conversion results clearly in tabular format.

Prompt:

"Write a Python program that connects to a free Currency Exchange API (like ExchangeRate-API) to convert a user-provided amount in Indian Rupees (INR) to USD, EUR, and GBP. Include input validation to ensure the amount is a positive number. Implement error handling for HTTP errors and API downtime. Display the final conversion rates and the calculated amounts in a clean, aligned tabular format."

Code:

```
import requests

def convert_currency(amount_inr):
    # Using a free, no-auth endpoint for exchange rates based on INR
    api_url = "[https://api.exchangerate-api.com/v4/latest/INR](https://api.exchangerate-api.com/v4/latest/INR)"
    target_currencies = ['USD', 'EUR', 'GBP']

    try:
        response = requests.get(api_url, timeout=10)
        response.raise_for_status()
        data = response.json()

        rates = data.get('rates', {})

        print("\n" + "="*45)
        print(f"{'Currency':<10} | {'Exchange Rate':<15} | {'Converted Amount':<15}")
        print("="*45)

        for currency in target_currencies:
            if currency in rates:
```

```

        rate = rates[currency]
        converted_amount = amount_inr * rate
        print(f"{currency:<10} | {rate:<15.4f} | {converted_amount:<15.2f}")
    else:
        print(f"{currency:<10} | {'N/A':<15} | {'N/A':<15}")

print("="*45)

except requests.exceptions.HTTPError as http_err:
    print(f"HTTP Error occurred: {http_err} - The API might be down or rate-limited.")
except requests.exceptions.ConnectionError:
    print("Network Error: Unable to reach the Exchange Rate API.")
except requests.exceptions.Timeout:
    print("Timeout Error: The request took too long to complete.")
except Exception as e:
    print(f"An unexpected error occurred: {e}")

if __name__ == "__main__":
    while True:
        try:
            user_input = input("Enter amount in INR to convert: ")
            amount = float(user_input)
            if amount < 0:
                print("Please enter a positive amount.")
                continue
            break
        except ValueError:
            print("Invalid input! Please enter a valid numerical amount.")

    convert_currency(amount)

```

Output:

Enter amount in INR to convert: 5000

```

=====
Currency | Exchange Rate | Converted Amount
=====
USD      | 0.0120         | 60.00
EUR      | 0.0111         | 55.50

```

GBP | 0.0095 | 47.50

=====

Task 3: GitHub Repository Info Fetcher

Task Description:

Connect to the GitHub API to fetch details of a repository. Handle rate limit errors, 404 (repo not found), and invalid input. Display repository name, description, stars, forks, and open issues.

Prompt:

"Create a Python script that uses the requests library to fetch information about a specific public GitHub repository using the GitHub REST API (<https://api.github.com/repos/{owner}/{repo}>). Extract and print the repository name, description, star count, fork count, and number of open issues. Implement comprehensive error handling for 404 (repository not found), API rate limits exceeded (403), and general network errors."

Code:

```
import requests

def fetch_github_repo_info(owner, repo):
    url = f"[https://api.github.com/repos/](https://api.github.com/repos/{owner}/{repo})"

    # Adding a generic User-Agent header is best practice for GitHub API
    headers = {
        "Accept": "application/vnd.github.v3+json",
        "User-Agent": "Python-Requests-App"
    }

    try:
        response = requests.get(url, headers=headers, timeout=10)

        if response.status_code == 404:
            print(f"Error: Repository '{owner}/{repo}' not found.")
            return
        elif response.status_code == 403:
            # Check if it's a rate limit error
```

```

        if 'X-RateLimit-Remaining' in response.headers and response.headers['X-RateLimit-Remaining'] == '0':
            print("Error: GitHub API rate limit exceeded. Please try again later.")
        else:
            print("Error: Access forbidden. You may need authentication.")
        return

    response.raise_for_status()
    repo_data = response.json()

    print("\n--- GitHub Repository Details ---")
    print(f"Name      : {repo_data.get('full_name', 'N/A')}")
    print(f"Description : {repo_data.get('description', 'No description provided.')}")
    print(f"Stars      : {repo_data.get('stargazers_count', 0)}")
    print(f"Forks      : {repo_data.get('forks_count', 0)}")
    print(f"Open Issues : {repo_data.get('open_issues_count', 0)}")
    print("-----")

except requests.exceptions.ConnectionError:
    print("Error: Failed to connect to GitHub API. Check your network.")
except requests.exceptions.Timeout:
    print("Error: The request to GitHub timed out.")
except Exception as e:
    print(f"An unexpected error occurred: {e}")

if __name__ == "__main__":
    repo_input = input("Enter the GitHub repository (format: owner/repo): ")
    try:
        owner_name, repo_name = repo_input.strip().split('/')
        fetch_github_repo_info(owner_name, repo_name)
    except ValueError:
        print("Invalid format. Please enter as 'owner/repo' (e.g., torvalds/linux).")

```

Output:

Enter the GitHub repository (format: owner/repo): octocat/Hello-World

```

--- GitHub Repository Details ---
Name      : octocat/Hello-World
Description : My first repository on GitHub!

```

Stars : 2451
Forks : 2102
Open Issues : 158

Task 4: Real-Time Application: News Headlines Aggregator

Task Description:

Build a News Aggregator using a free News API. Fetch the top 5 headlines for a given category (sports, technology, health). Handle errors like invalid category, missing API key, and request failures. Display headlines in a numbered list and include a retry mechanism if the first request fails.

Prompt:

"Write a Python script to build a News Aggregator using the NewsAPI (or a similar service). The program should ask the user for a category (e.g., technology, sports, health) and fetch the top 5 current headlines. Implement a retry mechanism using a loop that attempts the request up to 3 times with a short delay if it fails due to network issues. Include error handling for invalid categories and unauthorized API keys. Display the headlines in a formatted numbered list."

Code:

```
import requests
import time

def fetch_top_headlines(category):
    api_key = "YOUR_NEWS_API_KEY" # Replace with valid NewsAPI key
    url = "[https://newsapi.org/v2/top-headlines](https://newsapi.org/v2/top-headlines)"

    params = {
        'country': 'us',
        'category': category.lower(),
        'pageSize': 5,
        'apiKey': api_key
    }
```

```
valid_categories = ['business', 'entertainment', 'general', 'health', 'science', 'sports',  
'technology']
```

```
if category.lower() not in valid_categories:  
    print(f"Error: '{category}' is not a valid category.")  
    print(f"Valid options: {' '.join(valid_categories)}")  
    return
```

```
max_retries = 3  
retry_delay = 2 # seconds
```

```
for attempt in range(1, max_retries + 1):  
    try:  
        print(f"Fetching news... (Attempt {attempt}/{max_retries})")  
        response = requests.get(url, params=params, timeout=10)  
  
        if response.status_code == 401:  
            print("Error: Invalid API Key. Please authorize your request.")  
            return  
        elif response.status_code == 429:  
            print("Error: Too many requests. Rate limit exceeded.")  
            return
```

```
        response.raise_for_status()  
        data = response.json()
```

```
        articles = data.get('articles', [])  
        if not articles:  
            print(f"No headlines found for category: {category}")  
            return
```

```
        print(f"\nTop 5 Headlines in {category.capitalize()}:")  
        for idx, article in enumerate(articles, 1):  
            title = article.get('title', 'No Title')  
            source = article.get('source', {}).get('name', 'Unknown Source')  
            print(f"{idx}. {title} (Source: {source})")
```

```
        # Successful fetch, break out of the retry loop  
        break
```

```
except requests.exceptions.RequestException as e:
```



```
print(f"Request failed: {e}")
if attempt < max_retries:
    print(f"Retrying in {retry_delay} seconds...")
    time.sleep(retry_delay)
else:
    print("Max retries reached. Unable to fetch news at this time.")

if __name__ == "__main__":
    user_category = input("Enter a news category (e.g., technology, sports, health): ")
    fetch_top_headlines(user_category)
```

Output:

Enter a news category (e.g., technology, sports, health): technology
Fetching news... (Attempt 1/3)

Top 5 Headlines in Technology:

1. New AI chip announced with record-breaking processing speeds (Source: TechCrunch)
2. Major software update rolling out to smartphones globally (Source: The Verge)
3. Cybersecurity experts warn of new phishing campaign (Source: Wired)
4. Space agency successfully launches latest satellite into orbit (Source: Space.com)
5. Tech giant unveils its highly anticipated virtual reality headset (Source: CNET)